UltraBERT-JEV: Encoder models for System One Applications

Astarag Mohapatra

Independent Researcher

astarag2843@gmail.com

November 12, 2026

Abstract

We study encoder models that pick one of several options in a single forward pass, without generating text. This technical report compares recipes for fine-tuning UltraBERT-380M [32] with one mask per option. The four recipes differ in how an option is scored and how the scores become probabilities; one follows the expert gate of DeepSeek-V3 [14]. Trained on the same data as Qwen3-0.6B with the LLM-as-Jev recipe [27] (**67.6%** test accuracy), the encoder reaches **69.7%** to **72.1%**. Scoring an option by "yes" minus "no" instead of "yes" alone gives the larger gain, from 69.7% to 71.3% on the test set and from 57.8% to **59.6%** on external benchmarks (**56.1%** for the decoder). Sigmoid normalisation adds 0.8 test points; a square-root softplus adds none. One request at a time on an NVIDIA A100, the encoder takes **29 ms** per decision against **68 ms**, but uses more peak memory (2.8 against 2.2 GiB). An auxiliary head, trained on human-marked evidence and on labels selected with a reward adapted from SelfCite [7], predicts in the same pass the sentences annotated as support for the answer.

1 Introduction

Software often has to choose among fixed options: route a support ticket, classify an intent, or check a claim against evidence. A chat model can do this, but its answer must be generated and parsed. Jev-style decision models [2] instead return a probability distribution over the options, so a new task needs only a prompt and a list of options. LLM-as-Jev [27] shows that an unmodified decoder language model works as such a model: it reads numbered options and answers with a number. Fine-tuning it with KL anchors helps most for small models and for tasks with many options.

A decision needs almost no generation, while decoders are built for it. Multi-query and grouped-query attention [1, 38] shrink the KV cache for decoding, and the parameters they save can go into larger MLP layers. Du et al. [16] show that these layers cause memory spikes in workloads that only read their input. An encoder keeps no KV cache, which is one reason to try it; at our scale, Section 4.5 finds no peak-memory advantage.

An encoder also reads the input in both directions. A classifier head would fix the label set, but a masked language model can score any option by whether it fills a mask next to the option with "yes" or "no". Recent encoders such as ModernBERT [42] and UltraBERT [32] read the long inputs that decisions over documents need.

Reading in both directions also helps with evidence. A decoder serving many questions about one document usually reads the document first, so that its KV cache can be reused, as with prefix caching in vLLM [25] and RadixAttention in SGLang [50]. The cached document states then do not depend on the question. Our encoder reads the question and the document together.

Our contributions are:

- **An encoder decision model.** UltraBERT-380M, fine-tuned with one mask per option on the LLM-as-Jev training mixture with Nimble added (Section 3.1). It needs no new head and handles any number of options in one forward pass.
- **Training recipes.** Four ways to score and normalise the mask predictions, one following the DeepSeek-

V3 expert gate [14], compared on accuracy, sensitivity to option order and removal, label frequencies and calibration.

- **A comparison with decoders under matched data.** Qwen3-0.6B [34], fully fine-tuned on the same examples, with learning rates chosen on dev and results on a held-out test split, and two reproduced LoRA models [21] as references. We report accuracy, external accuracy, time and memory.
- **Predicting supporting evidence.** An auxiliary head that predicts the annotated evidence for the chosen option, trained on human-marked labels and on labels selected with a reward adapted from SelfCite [7], and compared with ContextCite [10] and AT2 [11].

2 Method

2.1 Input format

Figure 1 shows one row in both formats. The options are shuffled each time a row is used, so the position of the correct option carries no information.

Decoder. The prompt follows LLM-as-Jev: the state, the question, the options numbered [1] to [K], and an instruction to answer only with the number. The reply is filled in up to "Best answer: [", with thinking turned off, so each option k is the token sequence "k]" and the options form a trie. The decoder chooses the option whose whole sequence has the highest log-probability.

Encoder. The state and question are the same, the instruction asks to mark the correct option with "yes" and the others with "no", and each option is listed as "[k] `[M]` description", with a mask token in front. There is no chat template. One forward pass gives the logits at every mask, and option k is scored from the logits of "yes" and "no", each with a leading space, at its mask.

2.2 Loss functions

Decoder: trie loss with KL anchors. The tree loss is the negative log-probability of the gold option's path through the trie, each step normalised over the tokens legal there,

$$\mathscr{L}_{\text{tree}} = -\sum_t \log \frac{\exp z_t(g_t)}{\sum_{v \in \mathscr{V}_t} \exp z_t(v)}, \tag{1}$$

where z_t are the logits at step t, g_t the gold token and $\mathscr{V}_t$ the legal tokens. Three anchor terms keep the model close to its starting version p_0 [27]: $\mathscr{L}_{\text{mass}}$ keeps the total probability on the legal tokens, $\mathscr{L}_{\text{out}}$ keeps how the remaining probability is spread over the rest of the vocabulary, and $\mathscr{L}_{\text{pos}}$ keeps the next-token distribution at the start of the reply and after the closing bracket. They are KL divergences to p_0, computed over its 64 most likely tokens and one bucket for the rest. The loss is $\mathscr{L} = \mathscr{L}_{\text{tree}} + \lambda(\mathscr{L}_{\text{mass}} + \mathscr{L}_{\text{out}} + \mathscr{L}_{\text{pos}})$ with $\lambda = 1$.

Encoder recipes. Let $\ell_k(v)$ be the logit of token v at mask k, and let g be the gold option. A score s_k says how strongly the model accepts option k; the decision is the option with the highest score. Training raises the gold option's score relative to the other options, but no mask gets its own yes-or-no target. Thus, a chosen option can still prefer "no": for example, scores of -3 and -8 select the first option. We train the following four recipes:

- **Yes-logit softmax.** The score is the "yes" logit alone, and the options compete through one softmax:

$$s_k = \ell_k(yes), \qquad p_k = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}, \qquad \mathscr{L}_{\text{softmax}} = -\log p_g. \tag{2}$$

- **Yes-minus-no softmax.** The score measures the preference for "yes" over "no", followed by the same softmax:

$$s_k = \ell_k(yes) - \ell_k(no), \qquad p_k = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}}, \qquad \mathscr{L}_{\text{softmax}} = -\log p_g. \tag{3}$$

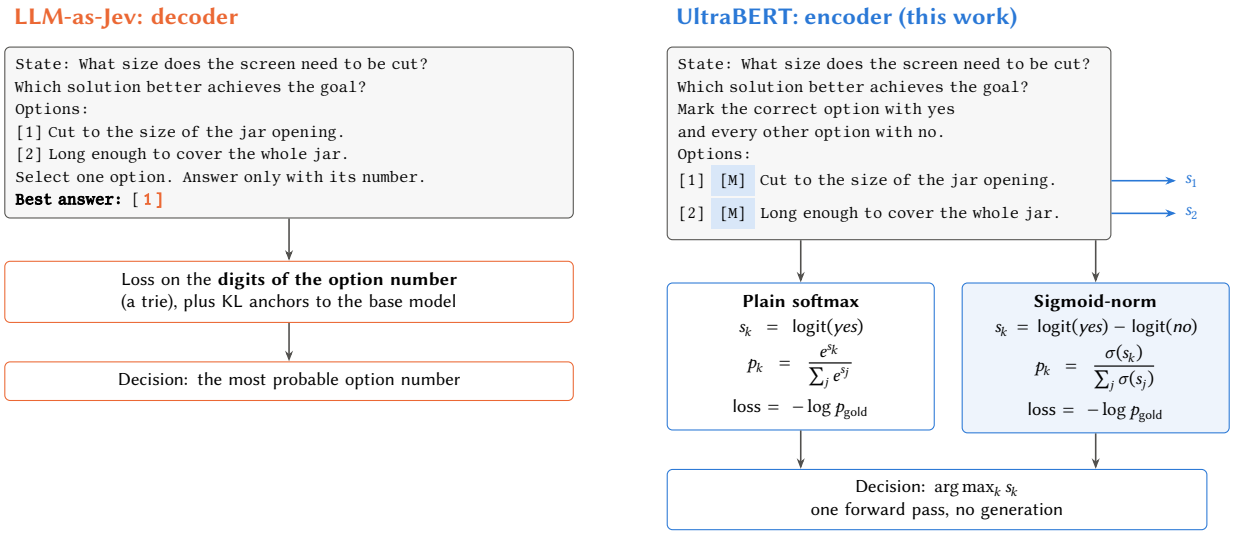


Figure 1. One row, two ways. The decoder reads the numbered options and writes the number of its answer; the loss is taken over the tokens of that number. The encoder gets a mask in front of each option and scores each option directly. Two of the four recipes are shown: the plain softmax over the options, which scores an option by the "yes" logit alone, and the sigmoid-normalised loss, which scores it by "yes" minus "no". Each option's score goes through a sigmoid, and the sum in the denominator keeps the options competing. The other two normalise "yes" minus "no" with a softmax and with a square-root softplus.

- **Yes-minus-no sigmoid normalisation.** Each score first goes through a sigmoid, and the resulting values are normalised by their sum:

$$s_k = \ell_k(yes) - \ell_k(no), \qquad p_k = \frac{\sigma(s_k)}{\sum_{j=1}^{K} \sigma(s_j)}, \qquad \mathcal{L}_{\text{sig-norm}} = -\log p_g. \tag{4}$$

  This is the form of the DeepSeek-V3 expert gate [14], with options in place of experts, normalised over all K options and with no selection or balancing bias. Each $\sigma(s_k)$ lies between 0 and 1, but only the ratios enter p_k, so the options still compete; the decision is still the highest score. The gradient with respect to s_k is

$$(1 - \sigma(s_k))\,(p_k - \mathbf{1}[k = g])\,. \tag{5}$$

  When an option's sigmoid approaches 1, the direct gradient of the loss with respect to its score becomes small. We exclude the sigmoid on the "yes" logit alone: vocabulary logits are around 200 (Section 5.3), so every sigmoid would saturate near 1 and the gradient would be negligible.

- **Yes-minus-no square-root softplus.** As an unbounded alternative to the sigmoid, we use $h(s) = \sqrt{\log(1 + e^s)}$:

$$s_k = \ell_k(yes) - \ell_k(no), \qquad p_k = \frac{h(s_k)}{\sum_{j=1}^{K} h(s_j)}, \qquad \mathcal{L}_{\text{sqrt-sp}} = -\log p_g. \tag{6}$$

  For a clearly rejected option ($s \ll 0$), $h(s) \approx e^{s/2}$, so the options behave as in a softmax at twice the temperature. For a clearly accepted option ($s \gg 0$), $h(s) \approx \sqrt{s}$ keeps growing, whereas the sigmoid stops at 1. The direct gradient with respect to its score therefore shrinks slowly, where the sigmoid's nearly vanishes.

3 Experimental setup

3.1 Training data

We train on the Reason+Long mixture of LLM-as-Jev [27], with Nimble added: 13 datasets covering intent routing (CLINC150 [26], MASSIVE [17], Bitext [6]), short reasoning (ReClor [45], LogiQA 2.0 [28], CosmosQA [22], PIQA [5],

ANLI [29], ProofWriter [39]), long input (ContractNLI [23], MuSiQue [40], QuALITY [30]) and Nimble [4]. Each row is a state (the passages), a question and a list of options. For example, a MuSiQue row gives about twenty paragraphs and a multi-hop question, and asks whether the paragraphs contain enough information to answer it ("Yes" or "No").

We keep 1,000 training rows per dataset, 13,000 in total, after these filters:

1. **Duplicates.** Rows with the same input are collapsed, and removed if their answers differ.
2. **Leakage.** A training row is removed if its input or source document appears in the validation split.
3. **Benchmark overlap.** A training row is removed if it shares a 13-word n-gram, or a sentence of six or more words, with an evaluation benchmark.
4. **Length.** Prompts over 10,240 tokens (Qwen3.5-4B tokenizer) are dropped, never truncated.

The held-out rows come from the validation split, with the same duplicate and length filters, up to 500 per dataset and in whole source documents (6,315 rows). A random 100 rows per dataset form the dev set (1,300 rows), used to choose learning rates and the evidence threshold; the other 5,015 rows form the test set, excluded from model selection. The split is by row, so a dev and a test row can share a source document, such as two statements about one contract.

3.2 Models

UltraBERT-380M [32] is fully fine-tuned with each of the four recipes of Section 2.2. Qwen3-0.6B [34] is fully fine-tuned with the decoder loss, with a frozen copy as p_0; it is the matched comparison. As references, we also report two LoRA models [21], Qwen3-0.6B and Qwen3-4B, reproduced from LLM-as-Jev on the same training set and loss.

3.3 Implementation details

Every model is trained for one epoch with a batch of 32 rows. We sweep the learning rate on dev, one run per value: $\{0.8, 1.5, 3, 5\} \times 10^{-5}$ for the encoder with the softmax over "yes", $\{0.8, 1.5, 2, 3\} \times 10^{-5}$ with the sigmoid-normalised loss, and $\{0.5, 1, 2\} \times 10^{-5}$ for the decoder. Both encoder sweeps choose 1.5×10^{-5}, which we reuse for the other two recipes; for Qwen the two best rates differ by 0.1 points, and we use 1×10^{-5}. At the chosen rate we train each model several times, changing only the row order and option shuffling, and report the average.

The metric is accuracy, and ALL is its mean over the 13 datasets. On the test set we average each row's correctness over a model's runs before computing ALL. For a difference between two models we give a paired 95% bootstrap interval: 2,000 resamples of the test rows within each dataset, with both models scored on the same rows. It covers test-row sampling only, not retraining, and treats rows from one document as independent, so it can be too narrow.

The encoder is scored with fp32 weights under bf16 autocast, since bf16 weights change its long-document results (Section 5.3); the Qwen checkpoints use bf16 weights.

3.4 External benchmarks and cost

To see how training changes behaviour outside the mixture, we score six benchmarks as decision rows, never used for any choice: BoolQ [8], MMLU [20], MMLU-Pro [41], ARC-Challenge [9], Winogrande [36] and SciQ [43]. We draw fixed random samples of 500, 1200, 800, 600, 500 and 400 rows, the sizes of LLM-as-Jev, whose samples are not released. External is the mean accuracy over the six.

Time per decision is the mean latency of one request over the dev rows, scored one at a time on one A100, as when a pipeline sends each item as it arrives. It covers the forward pass and option scoring but not tokenisation, after 20 warm-up rows, with the GPU synchronised around every call. The dev rows average 1,100 to 1,200 tokens. The encoder is timed on the softmax checkpoint and the decoders on the reproduced LoRA checkpoints with merged weights; the other checkpoints of each model share their architecture. Appendix A gives batched throughput.

Table 1. Matched comparison: average accuracy (%) over training runs. Test ALL is on the test set; Dev ALL and External are on the dev set and the six external benchmarks. Learning rates were chosen on dev. Bold: best below 1B parameters.

Model	Params	Dev ALL	Test ALL	External
UltraBERT-380M, softmax, yes	380M	71.2	69.7	57.8
UltraBERT-380M, softmax, yes − no	380M	71.7	71.3	**59.6**
UltraBERT-380M, sqrt-softplus, yes − no	380M	71.6	71.2	59.3
UltraBERT-380M, sigmoid-norm, yes − no	380M	**72.7**	**72.1**	59.5
Qwen3-0.6B, full FT	0.6B	69.0	67.6	56.1
Qwen3-0.6B, LoRA (reproduced)	0.6B	64.5	63.0	56.9
Qwen3-4B, LoRA (reproduced)	4B	79.3	77.3	73.5

4 Results

4.1 Encoder against decoder

Table 1 gives the main comparison. Every encoder recipe is above fully fine-tuned Qwen3-0.6B, both on the test set (69.7% to 72.1% against 67.6%) and on the external benchmarks (57.8% to 59.6% against 56.1%), with fewer parameters (380M against 0.6B). These results concern these two models under our setup; the reproduced Qwen3-4B LoRA model, at 77.3%, remains more accurate.

4.2 Full fine-tuning against LoRA

The reproduced Qwen3-0.6B LoRA model reaches 63.0% on the test set, 4.6 points below full fine-tuning, so full fine-tuning is the stronger baseline. Its external accuracy (56.9%) is not below that of the full fine-tune (56.1%).

4.3 Recipes

Changing the score from "yes" to "yes" minus "no", both with the softmax, adds 1.6 test points and raises external accuracy from 57.8% to 59.6%. Sigmoid normalisation adds a further 0.8 test points (95% interval [+0.2, +1.3], Section 3.3) and nothing on the external benchmarks.

This gain is concentrated in MuSiQue (69.8% against 63.0%), which contributes about 0.5 of the 0.8 points. On the other twelve datasets it averages 0.3 points and goes both ways, from +3.0 on Nimble to −2.4 on PIQA (Table 7).

The square-root softplus, which has no upper bound, does not reproduce the gain (71.2% against 71.3% for the softmax). The comparison does not show why: the two functions also differ in their gradients, and for rejected options the square-root softplus acts as a softmax at twice the temperature.

4.4 Sensitivity to option order and removal

On the dev rows we score each row in four random option orders and once with one wrong option removed, and count how often the answer changes (Table 2). With the raw scores fixed, no normalisation could change the winner, so a change means the encoder recomputed its scores after reading the new list: the test measures the trained model. The two softmax recipes isolate the score; the softmax and the sigmoid over "yes" minus "no" isolate the normalisation used in training.

The score does most of the work. "Yes" minus "no" raises external accuracy from 57.8% to 59.6%, lowers the answer changes on removal from 5.9% to 4.8%, and lowers the largest gradient norm from 842 to 139. Under reordering, the score and the sigmoid each remove about one point of answer changes (15.6%, 14.6%, 13.7%).

On top of the score, the sigmoid raises dev accuracy from 71.5% to 72.9% and lowers the calibration error from 0.027 to 0.023. It also reduces label bias. On the three datasets whose options are the same labels in every row (ContractNLI, MuSiQue and ProofWriter), we compare how often the model predicts each label with how often that label is correct. The total variation (TV) between the two is the share of predictions that would have to change label for them to

Table 2. Sensitivity to option order and removal, on the dev set. Each row is scored in four random option orders; an answer change under reordering means the chosen option is not the same in all four. One wrong option, never the chosen one, is removed at random from rows with at least three options, and the answer is compared over the remaining options. Dev accuracy is on the first random order, so it differs slightly from Table 1, which keeps the original order. TV: total variation between the predicted and the gold frequencies, over the three datasets with a fixed label set (label) and over option positions (position). Calibration error: expected calibration error of the top option's probability. $|s_k|$: size of the raw option scores. Each metric is averaged over datasets, then over training runs. Bold: best in the row, higher for accuracy and lower for answer changes, TV and calibration error; the gradient norm, the largest over all training steps and so a single batch's value, and $|s_k|$ are not ranked.

	Softmax, "yes"	Softmax, "yes" − "no"	Sqrt-softplus, "yes" − "no"	Sigmoid-norm, "yes" − "no"
Dev accuracy (%)	71.4	71.5	71.1	**72.9**
External accuracy (%)	57.8	**59.6**	59.3	59.5
Answer changes, options reordered (%)	15.6	14.6	16.6	**13.7**
Answer changes, one wrong option removed (%)	5.9	**4.8**	5.4	4.9
Largest gradient norm in training	842	139	104	190
Label frequencies vs. gold (TV)	0.167	0.213	0.154	**0.139**
Position frequencies vs. gold (TV)	0.050	0.045	0.049	**0.041**
Calibration error, top option	0.038	0.027	0.032	**0.023**
Mean $\lvert s_k \rvert$	60.5	3.0	12.3	7.4

match, so 0 means no bias towards any label. It is 0.167 for the softmax over "yes", 0.213 for the softmax over "yes" minus "no" and 0.139 for the sigmoid; for example, on MuSiQue the softmax models predict "Yes" far more often than it is correct (Section 5.2). TV measures bias, not accuracy: a model could predict each label exactly as often as it is correct and still assign it to the wrong rows. The gold option's sigmoid averages 0.049; for small sigmoid values $\sigma(s) \approx e^s$, so the normalised sigmoid then approximates a softmax.

The square-root softplus has the lowest maximum gradient norm (104) but changes its answer most often under reordering (16.6%).

4.5 Time and memory

A decoder must read the whole trie: every option's score includes the closing bracket and, from ten options on, the later digits. For a fair comparison we time the decoders with a faster scorer that reads the input once and then all trie nodes in one more forward pass on the shared KV cache, under a mask that lets each node see the input and its own earlier tokens. In fp32 its scores match those of the evaluation scorer within 10^{-4} (40 dev rows with ten or more options); in bf16, as timed, both choose the same option on 1,297 of the 1,300 dev rows. The evaluation scorer copies the cache for each node and takes 92 ms and up to 8.6 GiB on Qwen3-0.6B.

One row at a time, the encoder takes 29 ms per decision, against 68 ms for Qwen3-0.6B and 124 ms for Qwen3-4B (Table 3); reading the input alone takes the decoders 36 and 85 ms. Peak memory tells a different story. Qwen3-0.6B peaks at 2.2 GiB, against 2.8 GiB for the encoder with the fp32 weights it needs for its accuracy, or 1.9 GiB with bf16 weights, which lose accuracy (Section 5.3). At this scale the encoder is faster but has no peak-memory advantage; we measured peaks, not memory over time.

Table 3. Time and peak GPU memory per decision, scoring the dev rows one at a time on one A100. The decoders use bf16 weights and read the input, then the whole trie of option numbers in one more pass on the shared KV cache. Memory is what PyTorch allocates, in GiB, weights included; rows >4,096 tok.: mean peak over rows longer than 4,096 tokens; max: the largest peak over all rows.

Model	ms per decision	Weights	Peak, rows >4,096 tok.	Peak, max
UltraBERT-380M, fp32 weights	29	1.6	2.6	2.8
UltraBERT-380M, bf16 weights (lower accuracy)	26	0.8	1.5	1.9
Qwen3-0.6B	68	1.1	2.0	2.2
Qwen3-4B	124	7.6	8.9	9.3

5 Analysis

The numbers in this section are from the dev set, with one training run per setting, unless noted.

5.1 One change at a time

Table 4 changes one part of the recipe at a time. On these single runs, "yes" minus "no" adds 0.4 dev points and 2.0 external points, and the sigmoid a further 1.3 dev points, most of them on MuSiQue (64% to 76%). Single runs on 100 rows per dataset are noisy: here Nimble falls from 55% to 43% with the sigmoid, while on the test set the sigmoid-normalised model is 3.7 points above the softmax over "yes" on Nimble. The words "correct" and "incorrect" in place of "yes" and "no" did not help (70.6% against 71.1% for the single word, 70.1% against 71.5% for the difference).

5.2 Predicted labels

The recipes predict different labels. On MuSiQue, "Yes" is correct for 48% of the dev rows; the sigmoid-normalised models predict it for 47 to 60%, and the two softmax recipes for 64% and 69%. On ProofWriter, "Unknown" is correct for a third of the rows; the softmax never predicts it and the sigmoid-normalised models predict it for 2 to 12%, so it stays largely unlearned.

5.3 Numerical precision

The encoder's masked-token logits are around 200, where bf16 steps by 1 or 2 and options would tie or swap, so we compute the masked-token head, the scores and the normalisation in fp32. Even then, bf16 weights changed MuSiQue accuracy by up to 8 points, presumably through rounding elsewhere in the network; we did not trace where. We therefore keep fp32 weights under bf16 autocast. Re-scoring a Qwen checkpoint in another precision moved its dev accuracy by at most 0.8 points.

Table 4. Encoder variants on the dev rows, one training run each (accuracy, %). The first row is the softmax with the "yes" logit, the setting used for the softmax results elsewhere. Bold: best in each column.

Variant	Dev ALL	External	MuSiQue	ContractNLI	Nimble
Softmax, "yes"	71.1	57.1	57.0	75.0	**55.0**
Softmax, "yes" − "no"	71.5	59.1	64.0	78.0	48.0
Sigmoid-norm, "yes" − "no"	**72.8**	**59.4**	**76.0**	**81.0**	43.0
Softmax, "correct"	70.6	56.8	48.0	77.0	52.0
Softmax, "correct" − "incorrect"	70.1	58.0	54.0	76.0	46.0

6 Predicting supporting evidence

We add a small auxiliary head on the frozen encoder that highlights, in the same forward pass, the document sentences annotated as supporting the chosen option. For each document token it reads the token's hidden state and that of the chosen option's mask, and outputs an evidence score between 0 and 1; a sentence takes the highest score among its tokens. The score is not a calibrated probability, because positive tokens are up-weighted in training, and the head is not evaluated as an explanation of how the encoder decides.

The head is trained on human-marked evidence where it exists, in ContractNLI, MuSiQue, Nimble and ProofWriter. For the other rows, a labelling model proposes sets of sentences for the gold answer, and we keep the set with the best reward adapted from SelfCite [7] (Figure 2, Appendix B).

Comparison. We compare with the encoder's average attention, AT2 [11] and ContextCite [10] on the 889 test rows with gold evidence; the word-highlighting threshold is chosen on 267 dev rows. The head is trained on evidence labels, while AT2 and ContextCite are fitted to the model's response to ablations.

We report sentence average precision (AP) against the gold sentences, how often the top sentence is gold, and the time per row. For the baselines we also report LDS [10], the rank correlation between predicted and actual changes in the answer's probability when sentences are removed; we did not evaluate it for the heads.

AT2 and ContextCite ran on the softmax encoder, so the like-for-like comparison is the head on that encoder; the head on the sigmoid-normalised encoder also changes the encoder. The heads see the option the encoder chose, and a wrong choice is scored against the gold option's evidence, so Table 5 measures the decision and the evidence together. The times are on these rows, so they cannot be subtracted from the decision time of Section 4.5.

Results. On the softmax encoder, the head has a higher sentence AP than ContextCite and AT2 (0.55 against 0.41 and 0.33), in 68 ms per row against 1,860 ms for ContextCite. The lead comes from Nimble and ProofWriter; ContextCite is better on ContractNLI and level on MuSiQue. The head on the sigmoid-normalised encoder reaches 0.68 and is level with or above ContextCite on those two datasets.

The words this head marks have a precision of 0.33 and a recall of 0.35, a rough guide rather than an exact span. Most of the accuracy comes from the human-marked labels: on the softmax encoder, a head trained on them alone reaches 0.53, and the generated labels raise it to 0.55 (Appendix B). Of the methods with an LDS, ContextCite follows the model most closely (0.64).

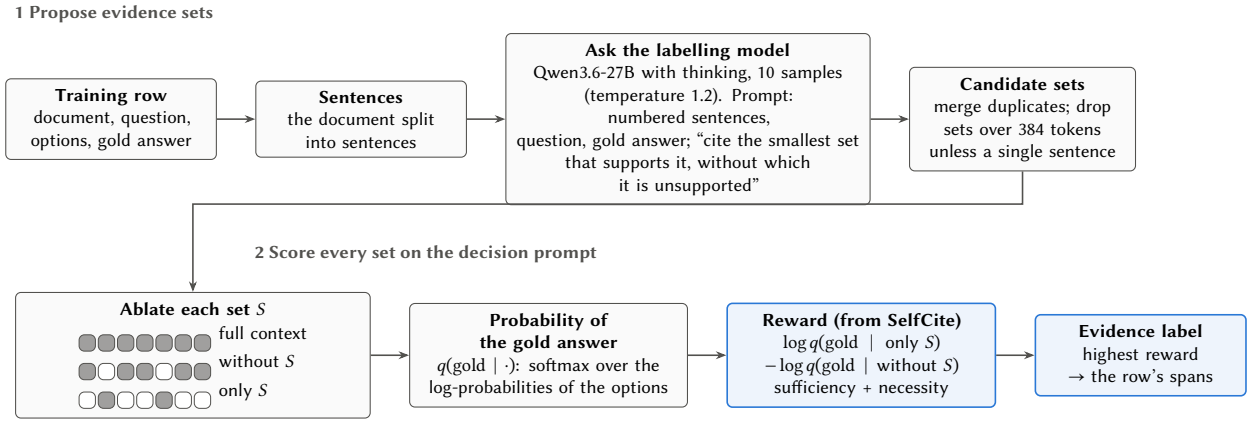


Figure 2. How evidence labels are generated for rows without human evidence. The labelling model, Qwen3.6-27B, proposes up to ten sets of sentences as evidence for the *gold* answer. The same model then answers the decision prompt three ways for each set S: with the full document, without S, and with only S: a good set is enough to keep the answer (sufficiency) and its removal loses the answer (necessity). The set with the highest reward becomes the label, which trains the evidence head. Unlike SelfCite, the answer scored is the gold answer and not the model's own response.

Table 5. Sentence evidence on the test rows with gold evidence. Sentence AP: average precision against the gold sentences, over all rows and per dataset (CNLI: ContractNLI, PW: ProofWriter). Top-1: share of rows where the top sentence is gold. LDS: how closely the scores follow the model's response to removed sentences; not evaluated for the heads. The first three methods and the first head use the encoder trained with the softmax. Each head is one training run. Bold: best in each column (lowest for time).

	Sentence AP							
Method	All	CNLI	MuSiQue	Nimble	PW	Top-1 (%)	LDS	ms per row
Average attention	0.25	0.07	0.20	0.34	0.38	14	0.09	69
AT2	0.33	0.30	0.28	0.30	0.42	23	0.15	71
ContextCite	0.41	0.52	0.33	0.33	0.44	44	**0.64**	1,860
Evidence head, softmax encoder	0.55	0.28	0.32	**0.96**	0.67	54	–	**68**
Evidence head, sigmoid-norm encoder	**0.68**	**0.55**	**0.52**	**0.96**	**0.68**	**74**	–	72

7 Related work

Jev-style decision models. Most community decision models compared in LLM-as-Jev [27] read the logits of an option letter or label from a decoder, frozen or tuned with LoRA; LLM-as-Jev studies this readout and when fine-tuning helps. AnyJev reads a typed decision from one prefill and corrects the label prior and option-order effects without training [46]. The Contrastive Language Model [24] embeds the state and each candidate separately with a frozen Qwen3-8B and scores them by cosine similarity, so candidates can be cached but never attend to the context. All of these start from a decoder; we start from an encoder that reads the context, the question and all options together.

Encoders that score options. Prompt-based classifiers use the masked-token head of an encoder with a verbalizer for a fixed set of labels [18, 37]. Two encoder decision models are listed on public Jev leaderboards [3]: Laya [12] and Von [31], both built on ModernBERT-large with one mask per option. Both add new layers, described in model cards: Laya trains its head with reinforcement learning on proper scoring rules, and Von uses a listwise softmax loss with a Brier term. We reuse the pretrained masked-token head, compare four recipes against decoders trained on the same data, and report a held-out test split.

Option letters, position and label bias. Reading the letter of the chosen option requires a model to bind the letter to the option text [35]. Models favour certain option labels, and their accuracy changes when the options are reordered [33, 49]. Corrections are usually applied after training, by calibrating with a content-free input [48] or by

dividing out a label prior and averaging over rotations of the options [46]. We shuffle the options during training and measure how the recipes differ in the labels they predict.

Encoders and decoders. Controlled studies pretrain encoders and decoders with the same recipe, and find that encoders do well on classification and retrieval [44] and that masked language modelling suits tasks that need text representations [19]. A much larger decoder can still beat an encoder on classification [47]. Decoder designs such as multi-query and grouped-query attention target long generation [1, 38], and Du et al. [16] study what they cost when nothing is generated (Section 1). Modern encoders keep long inputs and fast inference [32, 42].

Attribution and evidence. ContextCite learns a surrogate model from many ablations of the context [10]. AT2 learns to weight attention heads, and needs a decoder pass [11]. SelfCite fine-tunes a model to cite, using the change in the answer probability when the cited text is removed or kept alone as the reward [7]. Rationale benchmarks such as ERASER compare selected evidence with human rationales [15]. Our evidence head is trained on human-marked evidence and SelfCite-style labels, and gives the evidence in the decision's forward pass.

8 Conclusion

We fine-tuned an encoder, UltraBERT-380M, as a decision model with one mask per option, and compared four training recipes. On the same training examples, every recipe is more accurate than Qwen3-0.6B trained with LLM-as-Jev (69.7% to 72.1% against 67.6% on the test set). The encoder is faster (29 against 68 ms per decision) but uses more peak memory (2.8 against 2.2 GiB).

Among the recipes, scoring by "yes" minus "no" gives the larger test gain and all of the external gain. Sigmoid normalisation adds 0.8 test points, mostly on MuSiQue, and brings label frequencies closer to the gold ones; the square-root softplus does not, and we cannot say which property of the sigmoid is responsible. An auxiliary head on the frozen encoder predicts annotated evidence in the same pass, and on the softmax encoder it ranks the annotated sentences higher than AT2 and ContextCite, except on ContractNLI.

The results have limits. They cover one encoder, one decoder family and one English training mixture, below one billion parameters; a 4B decoder is still more accurate except on the intent datasets. We did not evaluate whether the evidence head reflects what the model used, and we measured only peak memory.

Two directions follow: larger encoders, to see whether the advantage holds above one billion parameters, and training the decision and the evidence together, so that the evidence head can shape the encoder.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.13245.

[2] Diogo Almeida. Introducing system one models & Jev. TypeSafe AI blog, https://typesafe.ai/blog/introducing-system-one-models-and-jev, 2026.

[3] Benchmark Heaven. JevBench: Jev-class model benchmark (v1.4.2.2). https://benchmarkheaven.com/jev-models/, 2026.

[4] Bespoke Labs. Nimble. https://github.com/bespokelabsai/nimble, 2025.

[5] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of AAAI*, 2020.

[6] Bitext. Bitext customer support LLM chatbot training dataset. https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset, 2023.

[7] Yung-Sung Chuang, Benjamin Cohen-Wang, Shannon Zejiang Shen, Zhaofeng Wu, Hu Xu, Xi Victoria Lin, James Glass, Shang-Wen Li, and Wen-tau Yih. SelfCite: Self-supervised alignment for context attribution in large language models. In *International Conference on Machine Learning*, 2025. arXiv:2502.09604.

[8] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of NAACL*, 2019.

[9] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[10] Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Madry. ContextCite: Attributing model generation to context. *arXiv preprint arXiv:2409.00729*, 2024.

[11] Benjamin Cohen-Wang, Yung-Sung Chuang, and Aleksander Madry. Learning to attribute with attention. *arXiv preprint arXiv:2504.13752*, 2025.

[12] Convai Innovations. Laya: an open non-autoregressive decision model. https://huggingface.co/convaiinnovations/laya, 2026.

[13] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

[14] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[15] Jay DeYoung, Sarthak Jain, Nazneen Fatema Rajani, Eric Lehman, Caiming Xiong, Richard Socher, and Byron C. Wallace. ERASER: A benchmark to evaluate rationalized NLP models. In *Proceedings of ACL*, 2020. arXiv:1911.03429.

[16] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, Ion Stoica, and Junchen Jiang. PrefillOnly: An inference engine for prefill-only workloads in large language model applications. *arXiv preprint arXiv:2505.07203*, 2025.

[17] Jack FitzGerald et al. MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages. In *Proceedings of ACL*, 2023.

[18] Tianyu Gao, Adam Fisch, and Danqi Chen. Making pre-trained language models better few-shot learners. In *Proceedings of ACL*, 2021. arXiv:2012.15723.

[19] Hippolyte Gisserot-Boukhlef, Nicolas Boizard, Manuel Faysse, Duarte M. Alves, Emmanuel Malherbe, André F. T. Martins, Céline Hudelot, and Pierre Colombo. Should we still pretrain encoders with masked language modeling? *arXiv preprint arXiv:2507.00994*, 2025.

[20] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021.

[21] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[22] Lifu Huang, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Cosmos QA: Machine reading comprehension with contextual commonsense reasoning. In *Proceedings of EMNLP-IJCNLP*, 2019.

[23] Yuta Koreeda and Christopher D. Manning. ContractNLI: A dataset for document-level natural language inference for contracts. In *Findings of EMNLP*, 2021.

[24] Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré, and Azalia Mirhoseini. Contrastive language models: A system one model for fast and generalizable decision-making. Blog post, Stanford University and NVIDIA Research, 23 September 2026. https://contrastive-lm.notion.site, 2026.

[25] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[26] Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill, Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. An evaluation dataset for intent classification and out-of-scope prediction. In *Proceedings of EMNLP-IJCNLP*, 2019.

[27] Yinheng Li and Justin Wagle. LLM-as-Jev: LLMs are already Jev-style decision models—when and how to fine-tune them. *arXiv preprint arXiv:2610.02076*, 2026.

[28] Hanmeng Liu, Jian Liu, Leyang Cui, Zhiyang Teng, Nan Duan, Ming Zhou, and Yue Zhang. LogiQA 2.0—an improved dataset for logical reasoning in natural language understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023.

[29] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. Adversarial NLI: A new benchmark for natural language understanding. In *Proceedings of ACL*, 2020.

[30] Richard Yuanzhe Pang, Alicia Parrish, Nitish Joshi, Nikita Nangia, Jason Phang, Angelica Chen, Vishakh Padmakumar, Johnny Ma, Jana Thompson, He He, and Samuel R. Bowman. QuALITY: Question answering with long input texts, yes! In *Proceedings of NAACL*, 2022.

[31] Victor Hugo Panisa. Von: a non-autoregressive decision model. https://huggingface.co/wfzyx/von, 2026.

[32] Jungwoo Park, Taewhoo Lee, and Jaewoo Kang. UltraBERT: Beyond the limit of BERT pre-training. *arXiv preprint arXiv:2610.15028*, 2026.

[33] Pouya Pezeshkpour and Estevam Hruschka. Large language models sensitivity to the order of options in multiple-choice questions. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024. arXiv:2308.11483.

[34] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[35] Joshua Robinson, Christopher Michael Rytting, and David Wingate. Leveraging large language models for multiple choice question answering. In *International Conference on Learning Representations*, 2023. arXiv:2210.12353.

[36] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. In *Proceedings of AAAI*, 2020.

[37] Timo Schick and Hinrich Schütze. Exploiting cloze questions for few shot text classification and natural language inference. In *Proceedings of EACL*, 2021. arXiv:2001.07676.

[38] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[39] Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of ACL-IJCNLP*, 2021.

[40] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 2022.

[41] Yubo Wang et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

[42] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, et al. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.

[43] Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text*, 2017.

[44] Orion Weller, Kathryn Ricci, Marc Marone, Antoine Chaffin, Dawn Lawrie, and Benjamin Van Durme. Seq vs seq: An open suite of paired encoders and decoders. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2507.11412.

[45] Weihao Yu, Zihang Jiang, Yanfei Dong, and Jiashi Feng. ReClor: A reading comprehension dataset requiring logical reasoning. In *International Conference on Learning Representations (ICLR)*, 2020.

[46] Jiamu Zhang, Tianze Yang, Yucheng Shi, Evan Chen, Zixiang Nie, Kelly Wan, Liangjie Hong, Ninghao Liu, and Liang Wu. AnyJev technical report. *arXiv preprint arXiv:2610.00831*, 2026.

[47] Hang Zhao, Qile P. Chen, Yijing Barry Zhang, and Gang Yang. Advancing single and multi-task text classification through large language model fine-tuning. *arXiv preprint arXiv:2412.08587*, 2024.

[48] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *International Conference on Machine Learning*, 2021. arXiv:2102.09690.

[49] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. Large language models are not robust multiple choice selectors. In *International Conference on Learning Representations*, 2024. arXiv:2309.03882.

[50] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

A Implementation details

Table 6 lists the training settings. Both models train for one epoch, which is 406 steps at a batch of 32.

Table 6. Training settings. Learning rates are in Section 3.3.

	UltraBERT-380M	Qwen3-0.6B (full)
Optimiser	AdamW, $\beta = (0.9, 0.98)$, $\epsilon = 10^{-6}$	AdamW
Weight decay	0.1	0.01
Schedule	5% warmup, linear decay to 0	100 warmup steps, linear decay to 10%
Gradient clipping	1.0	1.0
Precision	fp32 weights, bf16 autocast	fp32 weights, bf16 autocast
Attention	PyTorch SDPA	PyTorch SDPA
Gradient checkpointing	yes	yes
Token limit	32,768	16,384

The encoder settings are those of the UltraBERT model card, and we change only the learning rate. No training row exceeds either token limit, because the data are cut at 10,240 tokens (Section 3.1).

Batched throughput. These numbers are specific to our implementations, not a comparison of the fastest paths. When the dev rows are scored in batches, the encoder processes 75.5 rows per second, against 16.4 for Qwen3-0.6B and 6.5 for Qwen3-4B. The encoder uses packed variable-length FlashAttention-2 [13] and the decoders use length-sorted padded batches with the cache-copying trie scorer, so part of this gap comes from the padding and the copies.

B Evidence head

We train one head on each fine-tuned encoder (sigmoid-normalised and softmax, both at a learning rate of $1.5 \cdot 10^{-5}$); the softmax one matches the encoder used by AT2 and ContextCite.

Model. For document token t and option k, the head reads $[h_t; h_k; h_t \odot h_k]$, the final hidden states of the token and of option k's mask. A two-layer perceptron (512 hidden units, GELU, dropout 0.1) maps it to one logit, whose sigmoid is the token's evidence score for option k.

Labels. A token is positive if it overlaps an evidence span. We use gold evidence where it exists (Nimble, ContractNLI, MuSiQue, ProofWriter) and generated spans elsewhere (Figure 2): ANLI, QuALITY, LogiQA 2.0, CosmosQA and the remaining rows of ContractNLI, MuSiQue and ProofWriter. There are 7,778 labelled training rows (7,390 for training, 388 for validation), with 5.2% positive tokens.

Generated spans. Qwen3.6-27B with thinking gets the numbered sentences, the question and the gold answer, and is asked for the smallest set of sentences that supports the answer and without which it is unsupported. We sample ten replies (temperature 1.2, top-p 0.9), merge identical sets, and drop sets over 384 tokens unless they are one sentence.

Reward. The same model scores the decision prompt with each set S removed and with only S kept; $q(\text{gold} \mid \cdot)$ is the softmax over the options' summed token log-probabilities. The label is the set with the highest reward $\log q(\text{gold} \mid \text{only } S) - \log q(\text{gold} \mid \text{without } S)$, which is sufficiency plus necessity, with the full-context term cancelling. The best set is always kept, so a row whose candidates are all poor still gets a label, and nothing beyond the token cap makes the set the smallest sufficient one. Unlike SelfCite, we score the gold answer, so the reward is informative only where the labelling model believes it. We did not check the generated labels against human evidence directly; the last paragraph of this appendix is an indirect check.

Training. Token-level binary cross-entropy, with positives weighted by the negative-to-positive ratio (18.1), on the frozen encoder. The head is conditioned on the gold option in training and on the chosen option at test time. AdamW (learning rate 10^{-3}, weight decay 0.01), 16 rows per step, three epochs; we keep the epoch with the best validation token average precision.

Evaluation. All methods are scored on the test rows with gold evidence and at least two sentences. A word is highlighted when its evidence score reaches a threshold tuned for word F1 on the 267 dev rows with gold evidence: 0.79 for the sigmoid-normalised encoder and 0.74 for the softmax encoder. On the sigmoid-normalised encoder, the head's token average precision is 0.46 (0.17 for a random ranking), and 0.63 of its ten highest tokens are gold. On the softmax encoder, adding the generated labels to the gold ones raises sentence average precision from 0.53 to 0.55 and token average precision from 0.38 to 0.39.

C Additional results

Table 7 gives the test accuracy for each dataset.

Table 7. Test accuracy (%) per dataset, averaged over training runs. UB is UltraBERT-380M: UB yes is the softmax over each option's "yes" logit; UB yes−no is the softmax over "yes" minus "no"; UB sqrt-sp and UB sig-norm normalise "yes" minus "no" with the square-root softplus and the sigmoid, each divided by the sum over options (Section 2.2). Bold: best below 1B parameters.

Dataset	Rows	UB yes	UB yes−no	UB sqrt-sp	UB sig-norm	Qwen full	Qwen LoRA	Qwen-4B LoRA
CLINC150	400	**91.8**	91.5	91.7	**91.8**	86.3	79.8	89.8
MASSIVE	400	83.7	84.5	83.8	**84.9**	76.9	68.8	83.5
Bitext	400	**99.3**	**99.3**	**99.3**	**99.3**	96.0	94.8	97.2
ReClor	400	58.9	60.7	**61.8**	59.9	57.7	51.2	72.2
LogiQA 2.0	400	52.9	54.8	55.4	54.8	**57.0**	56.0	73.5
CosmosQA	400	74.8	74.2	75.1	**75.2**	59.9	55.0	76.0
PIQA	400	74.8	**77.5**	75.9	75.1	70.2	68.8	83.8
ANLI	400	73.1	72.6	**73.9**	73.6	63.7	61.0	74.0
ProofWriter	400	60.7	61.5	61.4	61.6	**63.3**	63.0	70.5
ContractNLI	393	73.8	77.4	74.7	**78.6**	76.1	65.6	79.1
MuSiQue	400	58.9	63.0	62.2	**69.8**	62.9	54.8	71.5
QuALITY	398	56.9	**62.1**	60.8	61.3	51.5	50.0	68.8
Nimble	224	47.0	47.8	49.1	50.7	**57.3**	50.9	64.3

D Examples

Each example shows the sigmoid-normalised encoder's decision and its evidence head's token scores, on the first test row of the evidence study with at most 1,400 tokens in each dataset (MuSiQue has none). Tokens at or above the dev-tuned threshold of 0.79 are highlighted, darker for a higher score, and gold evidence is underlined. The encoder is correct on all three rows, which were picked by this rule, not by outcome.

ContractNLI. 1,338 tokens. The model chooses option 1, which is **correct**. Question: Statement: "Receiving Party shall not reverse engineer any objects which embody Disclosing Party's Confidential Information." Does the contract entail the statement, contradict it, or not mention it?

k	Probability	Option	
1	85%	The contract entails the statement.	gold, chosen
2	0%	The contract contradicts the statement.	
3	15%	The contract does not mention it.	

Contract MUTUAL NONDISCLOSURE AGREEMENT Effective Date: 12/10/98 This Agreement governs the disclosure of information by and between Yahoo! Inc., a California corporation, and Restrac, Inc. ("Participant"). 1. The "Confidential Information" is that confidential, proprietary, and trade secret information being disclosed by the disclosing party described as (please be specific): (a) Yahoo Confidential Information (owned by Yahoo and any of its affiliates): Resume and Employment Related. (b) Participant Confidential Information: Resume and Employment Related. 2. Except as set forth in this Section 2, all Confidential Information shall be in tangible form and shall be marked as Confidential or proprietary information of the disclosing party. If the Confidential Information is disclosed orally or visually, it shall be identified as such at the time of disclosure and confirmed in a writing to the recipient within thirty (30) days of such disclosure. 3. Each of the parties agrees that it will not make use of, disseminate, or in any way disclose any Confidential Information of the other party to any person, firm or business, except to the extent necessary for negotiations, discussions and consultations with personnel or authorized representatives of the other party and any purpose the other party may hereafter authorize in writing. Each of the parties agrees that it shall disclose Confidential Information of the other party only to those of its employees who need to know such information and who have previously agreed, either as a condition to employment or in order to obtain the Confidential Information, to be bound by terms and conditions substantially similar to those of this Agreement. 4. There shall be no liability for disclosure or use of Confidential Information which is (a) in the public domain through no fault of the receiving party (b) rightfully received from a third party without any obligation of confidentiality, (c) rightfully known to the receiving party without any limitation on use or disclosure prior to its receipt from the disclosing party, (d) independently developed by the receiving party, (e) generally made available to third parties without any restriction on disclosure, or (f) communicated in response to a valid order by a court or other governmental body, as otherwise required by law, or as necessary to establish the rights of either party under this Agreement (provided that the party so disclosing has provided the other party with a reasonable opportunity to seek protective legal treatment for such Confidential Information). 5. "Residual Information" shall mean any Confidential Information of the disclosing party which may be retained in intangible form in the minds of those individuals of the receiving party who have had proper access to such Confidential Information. Notwithstanding anything else in this Agreement, the receiving party shall be free to use any Residual Information for any purpose whatsoever, including, without limitation, the development of its own products, provided that such party shall not be entitled to disclose Residual Information to any third parties unless such disclosure is in the course of, or as part of, any disclosure of its own products or their development. 6. Each of the parties agrees that it shall treat all Confidential Information of the other party with the same degree of care as it accords to its own Confidential Information and each of the parties represents that it exercises reasonable care to protect its own Confidential Information. 7. Each of the parties agrees that it will not modify, reverse engineer, decompile, create other works from, or disassemble any software programs contained in the Confidential Information of the other party unless otherwise specified in writing by the disclosing party. 8. All materials (including, without limitation, documents, drawings, models, apparatus, sketches, designs and lists) furnished to one party by the other, and which are designated in writing to be the property of such party, shall remain the property of

such party and shall be returned to it promptly at its request, together with any copies thereof. 9. This Agreement shall govern all communications between the parties that are made during the period from the effective date of this Agreement to the date on which either party receives from the other written notice that subsequent communications shall not be so governed, provided, however that each party's obligations under Sections 2 and 3 with respect to Confidential Information of the other party which it has previously received shall continue unless and until such Confidential Information falls within Sections 4 or 5. 10. Neither party shall communicate any information to the other in violation of the proprietary rights of any third party. Neither party acquires any licenses under any intellectual property rights of the other party under this Agreement. This Agreement shall be governed in all respects by the laws of the United States of America and by the laws of the State of California as such laws are applied to agreements entered into and to be performed entirely within California between California residents. This Agreement may only be changed by mutual agreement of authorized representatives of the parties in writing. All notices or reports permitted or required under this Agreement shall be in writing and shall be by personal delivery, telegram, telex, telecopier, facsimile transmission or by certified or registered mail, return receipt requested, and shall be deemed given upon personal delivery, five (5) days after deposit in the mail, or upon acknowledgment of receipt of electronic transmission. Notices shall be sent to the addresses set forth at the end of this Agreement or such other address as either party may specify in writing. IN WITNESS WHEREOF, the parties have executed this Agreement in duplicate as of the date first written above. <TABLE> <S> <C> YAHOO! INC. ———————— By: /s/ Matt Rowlen By: /s/ M. J. Fahey ————————- ———————— Printed Name: Matt Rowlen Printed Name: M. J. Fahey ———————— ———————— Title: Manager of Business Development Title: President ———————— ———————— Address: Yahoo! Address: Lexington, MA ———————— ————————- ———————— ————————- ———————— ———————— </TABLE>

Nimble. 558 tokens. The model chooses option 3, which is **correct**. Question: Apply this rubric: HIGH—route is correct, custody is intact, and the claimant supplies both a unique identifier and one undisclosed feature without a material conflict; release is permitted. MEDIUM—route is correct and the claim is plausible, but either required identifying element is missing or custody has a gap; hold for more evidence. LOW—route is wrong or a material item, date, location, or identifier conflicts; do not release. Weather, staff clothing, and similar incidental observations are distractors, not material conflicts.

k	Probability	Option	
1	6%	Do not release: LOW confidence because the route is incorrect or material facts conflict with the intake record.	
2	35%	Send to the Transit Depot Vault but hold the backpack: MEDIUM confidence because an identifying requirement is missing or custody is incomplete.	
3	60%	Send to the Transit Depot Vault and release to Mara: HIGH confidence because custody is intact and both required identifying elements match without material conflict.	gold, chosen

Context A navy canvas backpack was found on Route 18 bus 604 at 18:20 on 12 May. Transit operator Imani Cole sealed it as T-884 and delivered it to the civic hall receptionist, who transferred the unbroken seal to lost-property custodian Arun Vale without any gap in custody.

Facts The backpack sealed as T-884 was found in the transit vehicle Route 18 bus 604. The custody chain for backpack T-884 has no gap between Imani Cole sealing it and Arun Vale receiving it from the civic hall receptionist. The seal on backpack T-884 remained unbroken through Arun Vale's receipt of it. The serial patch recorded for backpack T-884 is a unique identifier for that backpack. The serial-patch value recorded for backpack T-884 in the intake log is SP-7742. Mara Venn supplied the serial-patch value SP-7742 when she filed her claim. The hidden-feature detail recorded for backpack T-884, a brass key inside a torn inner-lining pocket, was not publicly disclosed before Mara Venn supplied it. Mara Venn supplied a matching brass-key hidden-feature detail. All other material items, dates, times, locations, and identifiers Mara Venn supplied agree with the intake record.

Policy Policy routes property found in transit vehicles to the Transit Depot Vault, regardless of where it was handed in.

ProofWriter. 208 tokens. The model chooses option 1, which is **correct**. Question: Statement: "Gary is cold." Based only on the theory, is the statement true, false, or unknown?

k	Probability	Option	
1	83%	True: the theory implies the statement.	gold, chosen
2	0%	False: the theory implies that the statement is false.	
3	17%	Unknown: the theory implies neither the statement nor its negation.	

Theory Charlie is blue. Charlie is quiet. Dave is blue. Dave is cold. Dave is young. Erin is blue. Erin is quiet. Erin is white. Erin is young. Gary is cold. Gary is quiet. Gary is young. All quiet things are furry. If Charlie is cold then Charlie is young. White things are cold. All young things are nice. If Gary is nice then Gary is young. If something is young then it is nice. If something is furry and blue then it is white. If something is young and furry then it is nice.